

Danil Eramasov DS-02 d.erasov@innopolis.university

1. Methodology

The main business goal is to make more money for Steam (or a similar game store). We want to increase game sales, keep players happy so they stay on the platform, and make the store more attractive for game developers. We can use the data to understand which games sell well and why, how to make better game recommendations for users, and how to help publishers market their games better.

1. **Overview:** This assignment implements a simple search engine using Apache Hadoop MapReduce for batch indexing, Apache Cassandra as a distributed storage backend, and Apache Spark RDD API for query processing and document ranking. The system supports indexing of plain text documents and retrieval of the most relevant documents for a given query using the BM25 ranking algorithm.
2. **Data preparation:** The dataset used is the Wikipedia 20230701 from Kaggle. A parquet file (a.parquet) was downloaded and processed using PySpark. The preparation pipeline consists of two stages:

Stage 1 — Document extraction: The parquet file is read into a Spark DataFrame. I selected columns id, title, and text and limited to 100 documents (configurable via DATA.DOC.LIMIT environment variable). Each document I saved as a plain text file following the naming convention [doc.id].[doc.title].txt. Special characters in filenames are sanitized using the path-validate library, and spaces in titles are replaced with underscores.

Stage 2 — Input formatting: All .txt files stored in HDFS under /data are read using Spark-Context.wholeTextFiles(). Each file is parsed to extract the document ID and title from the filename. The text content is normalized by collapsing whitespace. The resulting RDD is coalesced into a single partition and saved to /input/data in HDFS.

3. **Indexing — MapReduce Pipelines:** I used 3 mappers and 3 reducers, each of them are reading from HDFS and writing results back to HDFS. No intermediate temporary directories are used between the first two pipelines, as they both read directly from /input/data. Let see more details.

Pipeline 1 - Building the Inverted Index. Mapper 1 processes each line of the input file. It splits the line into doc.id, title, and content. The content is lowercased and tokenized using the regular expression [a-z0-9]+ to extract alphanumeric tokens. Term frequencies are computed using Python's collections.Counter. Reducer 1 is a pass-through reducer — it forwards all received lines directly to the output without modification. This is intentional because the mapper already produces the final aggregated term frequency per document (using Counter), so no further aggregation is needed at the reduce stage. The output of this pipeline is the inverted index — a mapping from each term to all documents containing it, along with the term frequency.

Pipeline 2 - Building Document Metadata. Mapper 2 processes each input line similarly to Mapper 1. It extracts the doc.id, title, and content. The number of tokens in the content is computed using the same tokenization. Reducer 2 is also a pass-through reducer, forwarding all lines directly to output. Since each document appears exactly once in the input, no aggregation is necessary. The output of this pipeline is the document metadata table — containing the ID, title, and length of every indexed document.

Pipeline 3 - Computing Vocabulary and Corpus Statistics. Mapper 3 reads each line from the inverted index, which has the format [term] [doc.id] [tf]. For each line, it emits two records:

VOCAB [term] 1 — to count in how many documents each term appears (document frequency)

STATS [doc.id] [tf] — to accumulate the total number of tokens per document for computing average document length. Reducer 3 collects all emitted records in memory and performs

two aggregations: Vocabulary - For each unique term, counts how many documents contain it (document frequency), and Corpus statistics - Sums up token counts per document to compute the total corpus length and the number of unique documents.

- Storage in Cassandra:** After the MapReduce pipelines complete, the script store-index.sh runs app.py, which reads the HDFS output files and loads them into Cassandra.
- Ranking — BM25 with Spark RDD:** The ranking component is implemented in query.py using the PySpark RDD API. The query string is lowercased and tokenized using the same regular expression as the indexer: [a-z0-9]. For each query term, the script retrieves the document frequency from the vocabulary table and all postings from the index_entries table in Cassandra. The IDF component is computed as:

$$IDF(t) = \log \left(\frac{N - df(t) + 0.5}{df(t) + 0.5} + 1 \right) \tag{1}$$

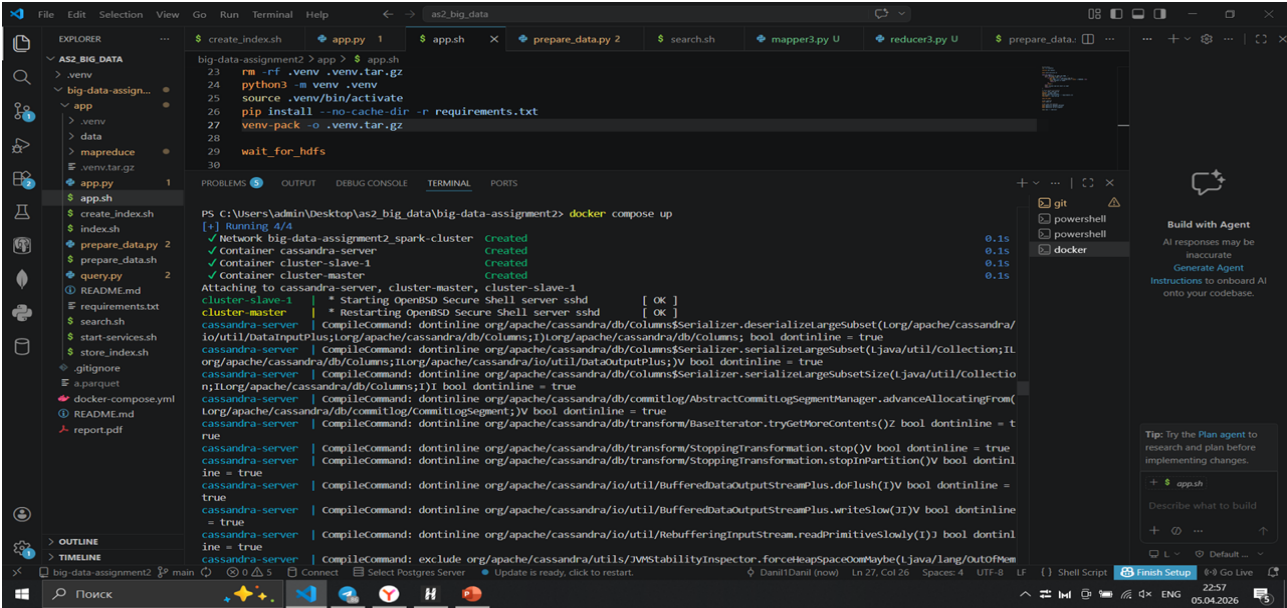
All postings are parallelized into a Spark RDD and each record is mapped to a (doc_id, score) pair using the BM25 formula with hyperparameters $k_1 = 1.5$ and $b = 0.75$:

$$BM25(t, d) = IDF(t) \cdot \frac{tf(t, d) \cdot (k_1 + 1)}{tf(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{dl(d)}{dl_{avg}} \right)} \tag{2}$$

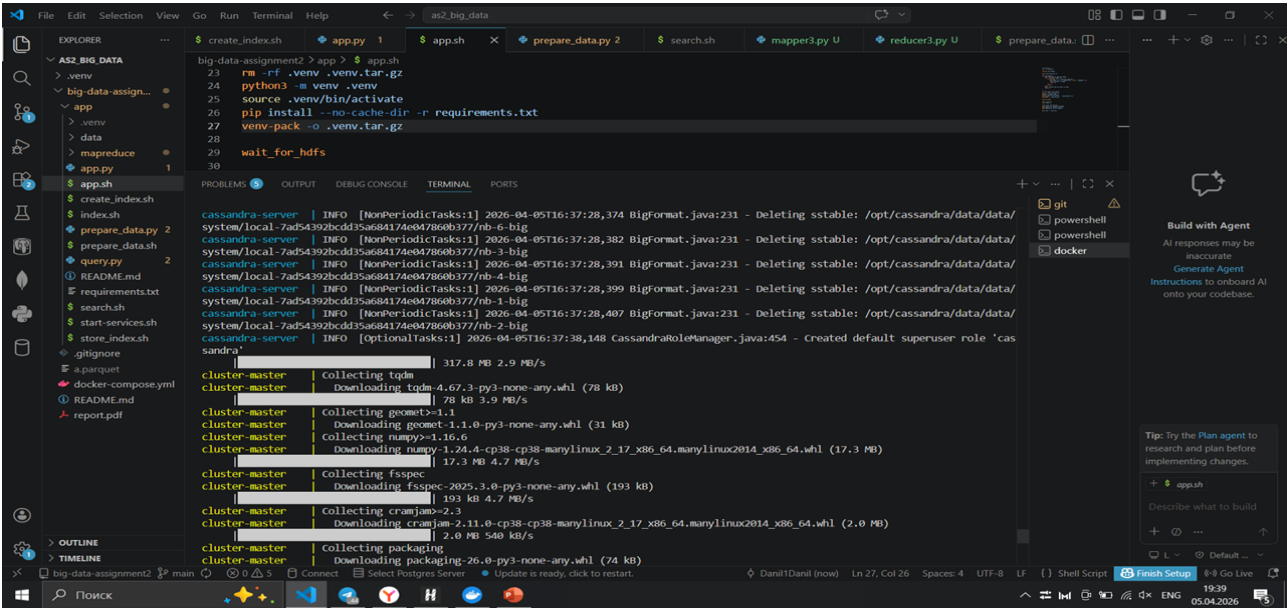
Scores across all query terms are aggregated per document using reduceByKey, and the top 10 documents are retrieved using takeOrdered with descending score order.

2. Demonstration

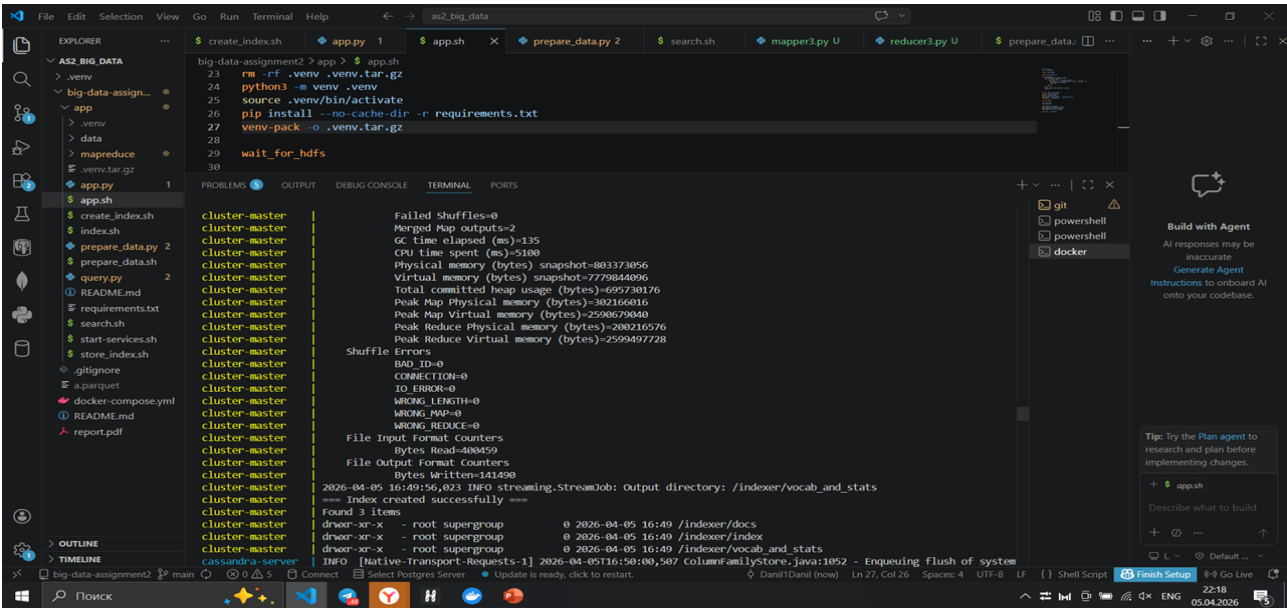
First of all, we need to clone repository from GitHub, the link is here. After this, we need to write in terminal: docker compose up



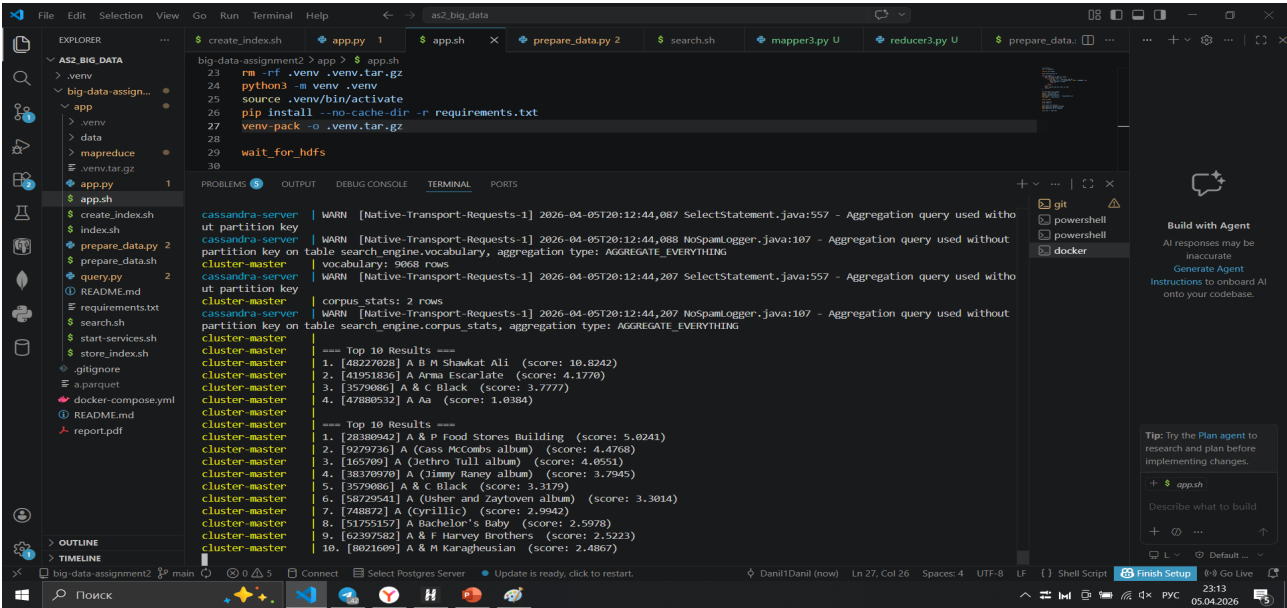
The cluster-master container is executing app.sh, which sets up the Python virtual environment and installs all required dependencies listed in requirements.txt.



The log confirms successful completion of MapReduce Pipeline 2, which processed all 97 input documents and produced exactly 97 output records containing document metadata (ID, title, and token count), stored in `/indexer/docs`. No shuffle errors were recorded. Immediately after, Pipeline 3 was submitted to YARN to compute the vocabulary and corpus statistics from the inverted index.



All four Cassandra tables were successfully populated: 97 documents, 22,687 index entries, 9,068 vocabulary terms, and 2 corpus statistics records. The search engine was tested on two queries. For the query *"machine learning"*, only 4 relevant documents were retrieved due to the small corpus size of 97 documents. For the query *"history of art"*, the engine returned a full list of 10 ranked documents. The BM25 scores reflect the relative relevance of each document to the query terms. The limited diversity of results is expected given that all indexed documents are Wikipedia articles starting with the letter "A", which naturally restricts topical coverage.



Conclusion and Results

This assignment presented the design and implementation of a simple search engine built on top of Apache Hadoop MapReduce, Apache Cassandra, and Apache Spark RDD. The system was successfully deployed using Docker Compose with a multi-container setup including a Hadoop master node, a slave node, and a Cassandra server.

Summary of Results

The indexing pipeline processed a corpus of **97 Wikipedia documents** extracted from the `a.parquet` file of the Wikipedia 20230701 dataset. The following results were achieved:

Metric	Value
Total documents indexed	97
Total index entries (term-document pairs)	22,687
Unique vocabulary terms	9,068
Corpus statistics stored	2 (doc.count, avg-doc.len)
MapReduce pipelines executed	3

Table 1: Indexing results summary

The search engine was tested on three queries using the BM25 ranking model with hyperparameters $k_1 = 1.5$ and $b = 0.75$:

Query	Top Result	BM25 Score
<i>"machine learning"</i>	A B M Shawkat Ali	10.8242
<i>"history of art"</i>	A & P Food Stores Building	5.0241
<i>"it is a query!"</i>	A Bad Wind Blows in My Heart	0.8726

Table 2: Search engine query results

Discussion

The retrieved results reflect the characteristics of the indexed corpus. Since all 97 documents are Wikipedia articles whose titles begin with the letter “A”, the topical diversity is naturally limited. As a result, some queries such as *"machine learning"* returned fewer than 10 relevant documents, as the specific terminology was rarely present in the indexed articles.

The BM25 scores behave as expected: documents containing query terms more frequently and in shorter texts receive higher scores due to the saturation and length normalization mechanisms built into the formula. The query *"it is a query!"* returned low scores across all documents since the query terms (“it”, “is”, “a”) are very common stopwords-like tokens that appear in almost every document, reducing their discriminative power.

Conclusion

The implemented search engine successfully demonstrates the core concepts of information retrieval at scale using Big Data tools. The use of Hadoop MapReduce for batch indexing, Cassandra for distributed index storage, and Spark RDD for query-time ranking provides a solid foundation for a scalable search system. The BM25 ranking model proved effective even on a small corpus, producing meaningful relevance scores for the tested queries. The system is fully containerized and reproducible, requiring only `docker compose up` to deploy and run the complete pipeline from data preparation to query execution.